WSL 2 + Docker + Dorarion

> Beta 数据库说明（2026-10-10）：当前 core 使用两个合并后的迁移文件。替换旧 Beta 数据库需要新的空数据库或明确删除旧数据后重新安装。不要只删除 SQLx 记录。请从当前源码构建新镜像，并参考 README 的最新政策；以前下载的 PDF 可能包含旧流程。

简易本地安装指南

> **目标：** 在 WSL 2 中安装 Ubuntu，安装 Docker，克隆 Dorarion，生成所需密钥，创建第一个管理员账户，然后在 Windows 浏览器中打开 Dorarion。

- Windows 命令请在以管理员身份运行的 PowerShell 中执行。
- Linux 和 Docker 命令请在 Ubuntu WSL 终端中执行。
- 本指南按照在本地计算机上实际成功使用的 Dorarion Compose 安装流程编写。
- 另有 PDF 版本。

1. 安装 WSL 2 和 Ubuntu

以管理员身份打开 PowerShell，然后运行：

```
wsl --install -d Ubuntu
```

如果系统提示，请重新启动 Windows。然后从开始菜单打开 Ubuntu，并创建 Linux 用户名和密码。

可在 PowerShell 中进行以下检查：

```
wsl -l -v
```

Ubuntu 的 `VERSION` 应显示为 2。

2. 准备 Ubuntu

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y git openssl ca-certificates curl
```

3. 安装 Docker Engine 和 Compose

3.1 删除旧的或冲突的 Docker 软件包

```
sudo apt remove -y docker.io docker-compose docker-compose-v2 \
  docker-doc docker-buildx podman-docker containerd runc
```

3.2 添加 Docker 官方软件源

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

3.3 安装并测试 Docker

```
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin

sudo docker run --rm hello-world
docker --version
docker compose version
```

下面的命令使用 sudo 运行 Docker。对于此本地安装没有问题，并且可以避免 Docker socket 权限问题。

4. 下载 Dorarion

```
mkdir -p ~/projects
cd ~/projects
git clone https://github.com/dorarion/dorarion-compose.git
cd dorarion-compose
```

5. 准备 Dorarion

先运行一次 Dorarion 的 Compose 安装脚本。该脚本会准备 Compose 环境，包括 `.env.docker`：

```
cp .env.docker.example .env.docker
chmod 600 .env.docker
${EDITOR:-nano} .env.docker
```

检查环境文件：

```
${EDITOR:-nano} .env.docker
```

如果 `.env.docker` 的所有者是 root，仅当编辑文件时出现权限错误，才运行：

```
sudo chown "$USER:$USER" .env.docker
```

6. 生成 Dorarion 密钥

在 `dorarion-compose` 目录中运行以下命令：

```
POSTGRES_PASSWORD="$(openssl rand -hex 32)"
JWT_SECRET="$(openssl rand -hex 32)"
SECRET_ENCRYPTION_KEY="$(openssl rand -base64 32)"
chmod 600 .env.docker

sed -i "s|^POSTGRES_PASSWORD=.*|POSTGRES_PASSWORD=$POSTGRES_PASSWORD|" .env.docker
sed -i "s|^JWT_SECRET=.*|JWT_SECRET=$JWT_SECRET|" .env.docker
sed -i "s|^SECRET_ENCRYPTION_KEY=.*|SECRET_ENCRYPTION_KEY=$SECRET_ENCRYPTION_KEY|" .env.docker
unset POSTGRES_PASSWORD JWT_SECRET SECRET_ENCRYPTION_KEY
```

确认没有剩余 placeholder 值：

```
if grep -q 'replace-' .env.docker; then echo 'Replace remaining image/credential placeholders'; fi
```

预期结果是没有任何输出。

> **请将 `.env.docker` 保密。** 该文件包含数据库和应用程序密钥。不要将其发布到聊天、截图、工单或 Git 提交中。

7. 启动 Dorarion

```
sudo ./scripts/install-compose.sh
sudo docker compose --env-file .env.docker ps
```

如果服务正在运行，请继续进行管理员设置。

8. 生成第一个 Dorarion 管理员密码

使用用户名 `admin` 和邮箱 `admin@example.com` 创建第一个本地管理员：

```
cd ~/projects/dorarion-compose
sudo docker compose --env-file .env.docker run --rm dorarion \
  bootstrap-admin admin admin@example.com
```

请立即将生成的管理员密码保存到密码管理器中。登录页面会使用此值。

字段	值
用户名	`admin`
邮箱	`admin@example.com`
密码	生成的 bootstrap 管理员密码

9. 打开 Dorarion

在 Windows 中打开 Chrome、Edge 或其他浏览器，然后访问：

http://127.0.0.1:3000/login

也可以使用：

http://localhost:3000/login

使用 `admin`（或 `admin@example.com`）以及第 8 步生成的管理员密码登录。

10. 常用检查命令

```
sudo docker compose --env-file .env.docker ps
sudo docker compose --env-file .env.docker logs --tail=100
```

如果 Dorarion 无法打开，请先检查容器状态是否为 `Up`，以及端口 `3000` 是否已公开。

快速总结

- 已安装 WSL 2 和 Ubuntu。
- 已安装 Docker Engine 和 Docker Compose。
- 已将 `dorarion-compose` 克隆到 `~/projects/dorarion-compose`。
- 已生成 `.env.docker` 密钥。
- 已启动 Dorarion 服务。
- 已使用 `bootstrap-admin` 创建第一个管理员。
- 已在 http://127.0.0.1:3000/login 打开 Dorarion。

官方平台参考资料

- 安装 Microsoft WSL
- 在 Ubuntu 上安装 Docker Engine

安装与升级 — reviewed core

这些设置针对已审查的 1.0 之前 core 源码。请选择由该源码构建的镜像；现有 旧版本 标签不能证明包含本次修复。

1.0 之前版本：请使用所选镜像附带的 API 契约。已审查源码支持 CI/CD、运维任务、事故及每日/每周活动报告。

- 一次生成 `JWT_SECRET`（至少 32 字节）和 `SECRET_ENCRYPTION_KEY`（32 个随机字节的 Base64），不要提交到 Git。
- API 连接池默认 15；每个副本还需预留 6 个 worker 连接及临时迁移连接。
- Compose 仅显式信任内部 `postgres` 主机；外部 PostgreSQL 需要 `sslmode=verify-full` 和可信 CA。
- 外部凭据使用 `env://DORARION_EXT_<NAME>`；core 秘密环境变量始终被拒绝。
- Argo CD 使用独立轮换的应用令牌，应用和集成都须启用。事故自动化使用限定仓库的 incident source。

/health/live 检查进程，/health/ready 检查 PostgreSQL。worker 状态单独通过需认证的 /health/workers 查看。Prometheus 使用 Secret 中 32–512 个 ASCII 字符的 DORARION_METRICS_SCRAPE_TOKEN，仅发送至 /metrics。接受转发的客户端地址前必须配置可信代理网络。

前向迁移前备份 PostgreSQL 和原始加密密钥。单独运行迁移 Job/服务，关闭服务器启动迁移。首次登录修改生成的管理员密码，升级后轮换 Argo 应用令牌。审计保留不能短于 365 天。每周报告覆盖发送前完整的本地 7 天；重试同一次手动发送应复用 Idempotency-Key。重新打开事故必须显式请求。演示端点已删除，未知请求字段被拒绝，请检查镜像内的 API 契约。上线前在自己的环境验证恢复、供应商消息和 readiness。

Compose: select DORARION_IMAGE before starting. Local HTTP uses DORARION_COOKIE_SECURE=false; set a reachable HTTPS public origin and secure cookies for remote use. The published port binds to 127.0.0.1 by default. /health/ready is database-only; the container uses the native dorarion-core healthcheck command. DORARION_MIGRATE_ON_START=false delegates migration to the one-shot service. The environment file is private (0600); avoid printing it.

发布与恢复注意事项（2026-10-27）

不要覆盖发布标签；此部署配置使用核心 0.0.8。将现有 .env.docker 中的 DORARION_IMAGE 改为 docker.io/kimc1992/dorarion-core:0.0.8@sha256:156e422514c7974010483a8938c579453d02f884353b9b4b9b9919537fed9f97。 这是已验证的共享镜像索引，包含 linux/amd64 和 linux/arm64；Docker 会自动选择匹配的平台。 替换整个镜像值，包括旧摘要。 确认 /health 和 /api/openapi.json 的版本均为 0.0.8。

提供者连接使用 DORARION_ALLOWED_PRIVATE_PROVIDER_HOSTS=gitlab.corp.example， Webhook 使用 DORARION_ALLOWED_PRIVATE_WEBHOOK_HOSTS=hooks.corp.example。 值为逗号分隔的精确主机名、IP 地址或 CIDR；不要包含 URL 协议、普通路径 或 gitlab.corp.example:8443 形式的主机名端口。提供者 URL 本身可以指定端口。 旧的全局允许开关已弃用。

停止应用与 receiver 写入，备份必须恢复到**新的空数据库**。保留原数据库 和加密密钥。不要使用 pg_restore --clean --if-exists 将旧备份覆盖到新版本 数据库；备份中没有的新对象可能残留，导致后续迁移失败。更改数据库连接， 执行迁移并确认 readiness 后再恢复流量。